

Figure 1: Profiling of HNSW indexing time. Distance computation constitutes the majority of the indexing time and consists of two components: memory accesses (B) and arithmetic operations (C). Other tasks (A), such as data structure maintenance, account for a minor portion.

In business scenarios, index rebuilding often occurs overnight during low user activity [108], constrained to a few hours. Serving approximately 100M vectors on a single node, the HNSW build time frequently exceeds 12 hours, failing to meet the requirement.

To enhance the index construction efficiency, employing specialized hardware like GPUs is a straightforward approach [92, 98]. By leveraging GPUs’ parallel computing capabilities, recent studies have parallelized distance computations and data structure maintenance for the graph indexing process on *million-scale* datasets [82, 115, 125], achieving an order of magnitude speedup over single-thread CPU implementations [115]. However, challenges such as high costs, memory constraints, and additional engineering efforts limit their practical applications [124]. For instance, even high-end GPUs like the NVIDIA A100, with tens of gigabytes of memory, struggle to accommodate large vector datasets requiring hundreds of gigabytes. As noted by Stonebraker in his recent survey [28], “If data does not fit in GPU memory, query execution bottlenecks on loading data into the device, significantly diminishing parallelization benefits.” Consequently, CPU-based deployment remains the prevalent choice in practical scenarios [42, 46, 100, 114], providing a cost-effective, memory-sufficient, and easy-to-use solution. This motivates us to explore optimization opportunities to improve the index construction efficiency on modern CPUs.

In this paper, we conduct an in-depth analysis of the HNSW construction process, identifying the root causes of inefficiency on modern CPUs. Figure 1 illustrates HNSW indexing time profiled using the *perf* tool [26] on two real-world embedding datasets [16, 24]. Results indicate that distance computation constitutes over 90% of the total construction time, marking it as the major bottleneck, while memory accesses and arithmetic operations contribute similarly. The lack of spatial locality in the HNSW index necessitates random memory accesses to fetch vectors for almost every distance computation. Building HNSW on a dataset of size n requires $O(n \log(n))$ distance calculations [78], leading to frequent cache misses as target data is often uncached. As a result, the memory controller frequently loads data from main memory, resulting in high access latency. Additionally, current vector data typically consists of high-dimensional floating-point numbers, occupying $4 \cdot D$ bytes for dimension D (e.g., 3KB for $D = 768$). However, Single Instruction, Multiple Data (SIMD) registers are restricted to hundreds of bits (e.g., 128 bits for the SSE instruction set) [21], much smaller than vector sizes. Thus, each distance computation requires hundreds of memory reads to load vector segments into a register, reducing the efficiency of SIMD operations. Notably, all distances are calculated solely for comparison during HNSW construction (refer to Section 2.2). Recent studies suggest that exact distances are unnecessary for comparison [50, 107, 113].

To address these issues, we optimize the HNSW construction process by reducing random memory accesses and effectively utilizing SIMD instructions. The theoretical analysis demonstrates that distance comparisons during index construction can be performed effectively using compact vector codes at an appropriate compression error. We initially implement mainstream vector compression methods—Product Quantization (PQ) [67], Scalar Quantization (SQ) [29], and Principal Component Analysis (PCA) [69]—into the HNSW construction process. However, these methods yield limited improvements in indexing efficiency or degrade search performance, as they do not align with HNSW’s construction characteristics. Further observations show that they fail to reduce excessive random memory accesses and do not leverage SIMD advantages. This motivates us to develop a specific compact coding strategy, alongside memory layout optimization for the construction process of HNSW, named Flash. To maximize SIMD instruction utilization, Flash is designed to accommodate SIMD register features, enabling concurrent execution of distance computations within the SIMD register. To minimize random memory accesses, Flash is effectively organized to match data access and register load patterns. Ultimately, Flash avoids numerous random memory accesses and enhances SIMD usage, achieving an order of magnitude speedup in construction efficiency while maintaining or improving search performance. To the best of our knowledge, this is the first work to reach such significant speedup on modern CPUs.

To sum up, this paper makes the following contributions.

- We deeply analyze the construction process of HNSW and identify the root causes of low construction efficiency on modern CPUs. Specifically, we find that distance computation suffers from high memory access latency and low arithmetic operation efficiency, which accounts for over 90% of indexing time.
- We propose a novel compact coding strategy, Flash, specifically designed for HNSW construction, optimizing index layout for efficient memory accesses and SIMD utilization. This approach improves cache hits and register-resident computation.
- We conduct extensive evaluations on real-world datasets, ranging from ten million to billion-scale, demonstrating that construction efficiency can be improved by over an order of magnitude, while search performance remains the same or even improves.
- We summarize three notable findings from our research: (1) a compact coding method that significantly enhances search performance may not be suitable for index construction; (2) reducing more dimensions may bring higher accuracy; and (3) encoding vectors and distances with a tiny amount of bits to align with hardware constraints may yield substantial benefits.

The paper is organized as follows: Section 2 reviews related work and conducts a problem study on index construction of HNSW. Section 3 introduces the proposed novel compact coding strategy alongside the memory layout optimization for HNSW construction. Section 4 provides experimental results, while some notable findings and conclusion are outlined in Sections 5 and 6, respectively.

2 BACKGROUND AND MOTIVATION

In this section, we first review current graph-based ANNS algorithms to ascertain the leadership of HNSW, then outline search

and indexing optimizations based on HNSW. Next, we provide a detailed analysis of the HNSW construction process to identify the root causes of inefficiency on modern CPUs.

2.1 Related Work

2.1.1 Graph-based ANNS Algorithms. Graph-based ANNS algorithms have emerged as the leading approach for balancing search accuracy and efficiency in high-dimensional spaces [38, 48, 66, 73, 88, 104]. These methods construct a graph index where vertices represent database vectors, and directed edges signify neighbor relationships. During search, a greedy algorithm navigates the graph, visiting a vertex's neighbors and selecting the one closest to the query, continuing until no closer neighbor is found, yielding the final result. Current methods share similar index structures and greedy search strategies but differ in their edge selection strategies during graph construction [104]. For example, HNSW [78] and NSG [49] use the Monotonic Relative Neighborhood Graph (MRNG) strategy, while Vamana [66] and τ -MG [88] incorporate additional parameters. HCNNG [81] employs the Minimum Spanning Tree (MST) approach, while KGraph [44] and NGT [64] implement the K-Nearest Neighbor Graph (KNNG) paradigm. Despite these variations, state-of-the-art methods like HNSW, NSG, Vamana, and τ -MG adhere to a similar construction framework, consisting of Candidate Acquisition (CA) and Neighbor Selection (NS) stages [49, 88]. Our research identifies the bottleneck in the CA and NS steps, where distance calculations dominate indexing time (see Section 2.2). Consequently, improving the CA and NS steps will accelerate the indexing process in all these graph algorithms. Notably, HNSW stands out for its robust optimizations and versatile features [39, 74, 128], including native *add* support. Numerous studies have enhanced HNSW from various perspectives (we review them in the following paragraphs), solidifying its role as a mainstream method in both industry and academia.

2.1.2 Search Optimizations on HNSW. Search optimizations for HNSW focus on several key components. Some aim to optimize entry point acquisition by leveraging additional quantization or hashing structures to start closer to the query [75, 126]. Others optimize neighbor visits by learning global [35] or topological [80] information, enhancing data locality [42], and partitioning neighbor areas [110]. These strategies improve accuracy and efficiency by identifying relevant neighbors and skipping irrelevant ones. Some approaches reduce distance computation complexity using approximate calculations [39, 50, 77, 113], enhancing search efficiency with minimal accuracy loss. Additionally, optimizations refine termination conditions through adaptive decision-making [71, 74, 112] and relaxed monotonicity [121]. Specialized hardware like GPUs [125], FPGAs [68, 87, 117], Compute Express Link (CXL) [65], Non-Volatile Memory (NVM) [91], NVMe SSDs [103], and SmartSSDs [97] accelerate distance computation and optimize the HNSW index layout, achieving superior performance through software-hardware collaboration. To support complex query processing, some approaches integrate structured predicates [85, 101] or range attributes [128] into HNSW, enabling hybrid searches of vectors and attributes. These optimizations enhance HNSW's performance and versatility, meeting diverse real-world requirements, though most increase indexing time compared to the original HNSW.

2.1.3 Indexing Optimizations of HNSW. Despite HNSW's superior search performance, its low index construction efficiency remains a major bottleneck, a challenge shared by other graph-based methods [33, 48, 49, 66, 75, 126]. As demands for dynamic updates [93, 111], large-scale data [40, 66], and complex queries [85, 128] grow (see Section 1), optimizing HNSW construction becomes increasingly important. Current research targets two areas: algorithm optimization and hardware acceleration. Algorithm optimization efforts have redesigned the construction pipeline, either fully [49, 88] or partially [75], but these attempts offer only marginal improvements [48, 49, 126]. Moreover, these optimizations substantially modify the algorithmic process of HNSW, requiring code refactoring that may disrupt many engineering optimizations integrated over the years [2–4]. Some HNSW features, like native *add* support, are weakened [75] or even discarded [49]. On the other hand, GPU-accelerated methods parallelize distance computations during HNSW construction [55, 82, 98]. These methods re-implement HNSW to align with GPU architecture, achieving an order of magnitude speedup compared to single-thread CPU-based construction [115]. However, GPU-specific limitations reduce their practicality in real-world scenarios [124] (see Section 1). As CPU-based HNSW remains prevalent in many industrial applications [42, 46, 100, 114], accelerating HNSW construction on modern CPUs is crucial. Note that parallel multi-thread implementations of graph indexing on CPUs are simple, efficient, and widely adopted by current graph indexes (e.g., HNSW, NSG [49], τ -MG [88]). These implementations are orthogonal to our work, as our techniques build upon multi-thread graph indexing.

2.1.4 Distributed Deployment of HNSW. Distributed deployment accelerates HNSW construction by partitioning large datasets into multiple shards, each containing tens of millions of vectors [43, 45, 56, 63]. This enables concurrent index construction across nodes, enhancing efficiency for large-scale datasets [45]. Query processing can be optimized by an inter-shard coordinator [56], and advanced data fragmentation strategies can assign queries to relevant shards [34, 120]. At the system level, distributed deployment ensures scaling, load balancing, and fault tolerance [103], but introduces challenges like high communication overhead [79, 100]. Notably, distributed deployment is orthogonal to our research, as we focus on efficient index building within a shard, which can be directly integrated into existing distributed systems.

2.2 Problem Analysis

Notations. In the HNSW index, each vertex corresponds to a unique vector, denoted by bold lowercase letters (e.g., $\mathbf{x}$, $\mathbf{y}$). We use the Euclidean distance $\delta(\mathbf{x}, \mathbf{y})$ as the distance metric for quantifying similarity, where a smaller $\delta(\mathbf{x}, \mathbf{y})$ indicates greater similarity [49, 88, 104, 126]. Bold uppercase letters (e.g., S) denote sets, while unbolded uppercase and lowercase letters represent parameters.

Given a vector dataset S , the HNSW index G is built by progressively inserting vectors into the current graph index, as outlined in Algorithm 1. Two hyper-parameters must be set before index construction: the maximum number of candidates (C) and the maximum number of neighbors (R)³. For each inserted vector $\mathbf{x}$, the

³In the original paper and many popular repositories [2, 4, 78], C is called *efConstruction*, and R in the base layer is twice that in higher layers [78].

Algorithm 1: INDEX CONSTRUCTION OF HNSW**Input:** a vector dataset S , hyper-parameters C and R ($R \leq C$)**Output:** HNSW index built on S

```

1  $V \leftarrow \emptyset, E \leftarrow \emptyset;$  ▷ start from a empty graph
2 for each  $x$  in  $S$  do ▷ insert all vectors in  $S$ 
3    $l_{max} \leftarrow x$ 's max layer; ▷ exponential decaying distribution
4   for each  $l \in \{l_{max}, \dots, 0\}$  do ▷ insert  $x$  at layer  $l$ 
5      $C(x) \leftarrow$  top- $C$  candidates; ▷ greedy search
6      $N(x) \leftarrow \leq R$  neighbors from  $C(x)$ ; ▷ heuristic strategy
7     add  $x$  to  $N(y)$  for each  $y \in N(x)$ ; ▷ reverse edge
8    $V \leftarrow V \cup \{x\}, E \leftarrow E \cup N(x)$ 
9 return  $G = (V, E)$ 

```

maximum layer l_{max} is randomly selected following an exponentially decaying distribution (line 3). The vector is inserted into layers from l_{max} to 0, with the same procedure applied at each layer (lines 4-7). At layer l , x is treated as a query point, and a greedy search on the current graph at layer l identifies the top- C nearest vertices as candidates (line 5). During the search, a candidate set $C(x)$ of size C is maintained, with vertices ordered in ascending distance to x , and the maximum distance is denoted as T . The search iteratively visits a vertex's neighbors, calculates their distances to x , and updates $C(x)$ with neighbors that have smaller distances ($< T$). To obtain the final neighbors, a heuristic edge selection strategy prevents clustering among neighbors (line 6). For example, for candidate v and any candidate u where $\delta(u, x) < \delta(v, x)$, if $\delta(u, v) < \delta(v, x)$, v is excluded from $N(x)$. After determining x 's final neighbors, x is added to the neighbor list $N(y)$ for each $y \in N(x)$ (line 7). If $N(y)$ exceeds R , the heuristic edge selection strategy prunes excess neighbors, ensuring that the neighbor count remains under R .

The construction process of HNSW involves two main steps: *Candidate Acquisition (CA)* and *Neighbor Selection (NS)*. The CA step identifies the top- C nearest vertices for an inserted vector using a greedy search strategy. This involves visiting a vertex's neighbor list and computing distances to the inserted vector, requiring random memory access to fetch a neighbor's vector data, stored separately from neighbor IDs due to the large vector size. In the NS step, the final R neighbors are selected from the candidate set using a heuristic strategy. Here, distances among candidates are computed, also requiring random access to their vector data. In both CA and NS, distance calculations are used solely for comparison. In CA, distances are compared to the largest distance in the candidate set to determine if a visiting vertex can replace the farthest candidate. In NS, distances among candidates and between candidates and the inserted vector are compared to decide the inclusion of a candidate in the final neighbor set. HNSW currently performs all distance computations on full-precision vectors. Additionally, to utilize SIMD acceleration, a distance computation requires hundreds of read operations to load vector segments into a 128-bit register due to a large vector size, often spanning thousands of bytes.

EXAMPLE 1. In Figure 2, we illustrate the index construction process of HNSW at the base layer, exemplified by inserting v_{18} with $C = 4$ and $R = 2$. In the CA step, v_{18} is treated as a query point, and a greedy search from v_0 is initiated. After examining v_0 's neighbors, the candidate set $C(v_{18})$ is $\{v_5, v_9, v_0, v_{13}\}$, ordered by proximity

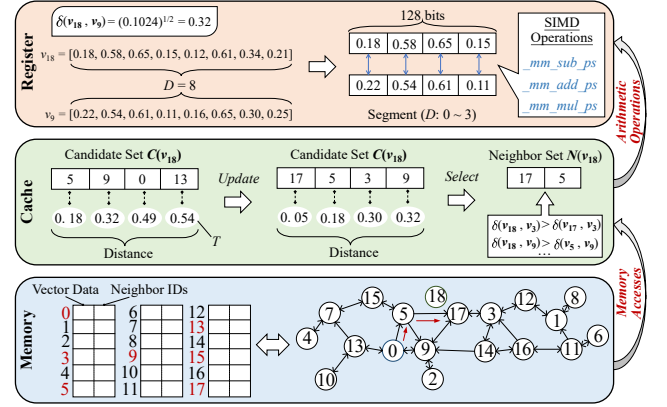


Figure 2: Illustrating memory accesses and arithmetic operations during the HNSW construction process (base layer).

to v_{18} . The current distance threshold T is $\delta(v_{13}, v_{18}) = 0.54$. The nearest vertex, v_5 , is then visited, prompting an exploration of its neighbors to update $C(v_{18})$. Upon calculating $\delta(v_{15}, v_{18})$ and comparing it with T , v_{13} is replaced by v_{15} as $\delta(v_{15}, v_{18}) < T$, and T is updated to $\delta(v_0, v_{18}) = 0.49$. This process continues, resulting in the replacement of v_0 and v_{15} with v_{17} and v_3 , respectively. At the end of CA, $C(v_{18})$ comprises $\{v_{17}, v_5, v_3, v_9\}$. The vertices $v_0, v_3, v_5, v_9, v_{13}, v_{15}$, and v_{17} are traversed in this phase. As shown in the graph index's memory layout (lower left), these vertices (highlighted in red) exhibit a random distribution, meaning numerous random memory accesses. In the NS step, v_{17} is initially chosen, leading to the removal of v_3 and v_9 from $C(v_{18})$ based on the conditions $\delta(v_{17}, v_3) < \delta(v_3, v_{18})$ and $\delta(v_{17}, v_9) < \delta(v_9, v_{18})$. The remaining vertex, v_5 , is added to the neighbor list, which reaches the limit $R = 2$, yielding $\{v_{17}, v_5\}$. v_{18} is then appended to the neighbor lists of v_{17} and v_5 , while ensuring that their neighbor counts remain below R through the heuristic strategy. In practice, $C(v_{18})$ can include thousands of candidates, making full vector caching impractical and necessitating many random memory accesses for distance computations during the NS stage. To leverage SIMD instructions, a vector is divided into multiple segments, each comprising four dimensions suitable for loading into a 128-bit register. While SIMD enhances processing speed, the necessity of numerous register load operations to sequentially fetch each vector segment leads to reduced SIMD utilization efficiency.

Our analysis reveals that the primary inefficiency in HNSW index construction lies in the current distance computation process, consuming over 90% of indexing time (Figure 1). This process suffers from numerous random memory accesses and suboptimal SIMD utilization, misaligning with modern CPU architecture. Thus, optimizing distance computation to better exploit modern CPU capabilities is essential for accelerating HNSW construction.

3 METHODOLOGY

In this section, we theoretically analyze how distance comparisons in HNSW construction can be effectively executed using compact vector codes. Based on this, we integrate three common vector compression methods—Product Quantization (PQ) [67], Scalar Quantization (SQ) [29], and Principal Component Analysis (PCA)